# ASSIGNMENT 5 – CO5

# RELIABLE SOFTWARE WITH RUNTIME POLYMORPHISM AND EXCEPTIONS

Hospital Billing System

| | |
|---|---|
| **Name** | J. Roopa Sai Divya |
| **Register Number** | 192524140 |
| **Course** | Object Oriented Programming with C++ |
| **Assignment** | Assignment 5 – CO5 |
| **Compiler** | G++ (C++17) |
| **GitHub Repository** | https://github.com/roopajallu814-arch/C-DSA0104-Roopa-192524140- |

## PART A – DESIGN (25 MARKS)

## TASK A1 – DESIGN THE POLYMORPHIC BILL HIERARCHY

### A1.1 – Identify the Common Interface

A hospital billing system handles different types of bills, such as Out-Patient, In-Patient, Day-Care Surgery and Emergency bills. The common base class is **Bill**. It contains the common data and operations required by every bill.

**Common data members:**

- Patient name or patient ID.
- Bill number.
- Common billing information.

**Common functions:**

- `total()` – calculates the total bill amount.
- `type()` – returns the type of bill.
- `print()` – displays the bill details.

### A1.2 – Design the Hierarchy

An abstract class is a class that contains at least one pure virtual function. It cannot be used to create objects directly. The `Bill` class is designed as an abstract base class. The derived classes implement the `total()` and `type()` functions.

**Derived classes:**

1. `OutPatientBill` – calculates consultation and test charges.
2. `InPatientBill` – calculates room charges, procedure charges and medicine charges.
3. `DayCareBill` – calculates the fixed package amount.
4. `EmergencyBill` – calculates the base charge and time-based charges.

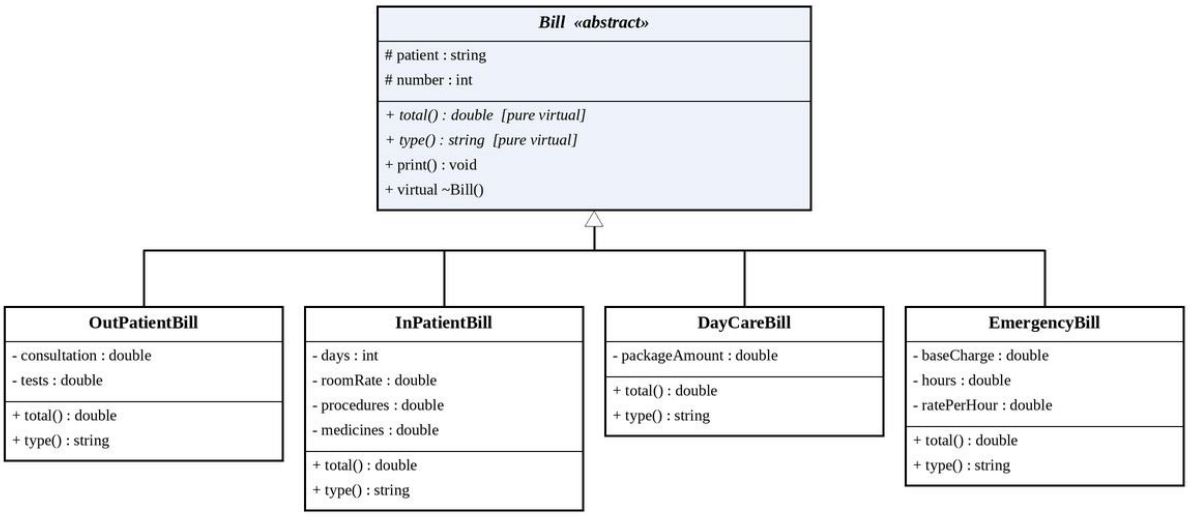



*Figure A1: UML class diagram of the Bill hierarchy*

**Explanation:**

- `Bill` is an abstract base class.
- `total()` and `type()` are pure virtual functions.
- `print()` is a normal member function that calls the virtual functions.
- Derived classes override the pure virtual functions.
- The virtual destructor ensures safe deletion through a base-class pointer.

## A1.3 – Process All Bills Together

**Pseudocode:**

```
BEGIN
    DECLARE array of Bill pointers
    DECLARE totalAmount = 0

    FOR each bill in the array
        DISPLAY bill details
        totalAmount = totalAmount + bill.total()
    END FOR

    DISPLAY totalAmount
END
```

### *Alternative Concept and Complexity Analysis*

If a single `Bill` structure with a type field and a `switch` statement is used, each bill is identified by its type. The `total()` function uses a `switch` statement to calculate the amount.

```
FUNCTION total(bill)
    SWITCH bill.type
        CASE OUTPATIENT:
            RETURN consultation + tests
        CASE INPATIENT:
            RETURN days * roomRate + procedures + medicines
        CASE DAYCARE:
            RETURN packagePrice
        CASE EMERGENCY:
            RETURN baseCharge + timeCharge
    END SWITCH
END FUNCTION
```

| Feature | Polymorphism | Switch statement |
|---|---|---|
| Design | Base and derived classes | One structure |
| Calculation | Virtual function | Switch case |
| Time complexity | O(n) for n bills | O(n) for n bills |
| Space complexity | O(1) extra per bill, excluding stored data | O(1) extra per bill, excluding stored data |
| Adding a new bill | Add a new derived class | Modify the switch statement |

**Conclusion:** Runtime polymorphism provides better extensibility because a new bill type can be added without changing the existing billing code.

## TASK A2 – UNDERSTAND RUNTIME BINDING

### A2.1 – Early and Late Binding

**Early binding** occurs during compile time. The compiler decides which function should be called before the program executes. It is commonly used with non-virtual functions.

**Late binding** occurs during runtime. The function call is decided based on the actual object type. It is achieved using virtual functions.

```
Bill* b = new OutPatientBill();
b->total();
```

If `total()` is virtual, the `OutPatientBill::total()` function is executed.

| Early Binding | Late Binding |
|---|---|
| Compile-time decision | Runtime decision |
| Uses non-virtual functions | Uses virtual functions |
| Faster function selection | Supports runtime polymorphism |
| Based on pointer/reference type | Based on actual object type |

### A2.2 – The Virtual Table (Vtable)

A vtable is a compiler-generated table that stores the addresses of the virtual functions of a class. Each object of a polymorphic class holds a hidden pointer (the vptr) to the vtable of its actual class.

| Class | total() | type() / Destructor |
|---|---|---|
| Bill | Pure virtual (no body) | Pure virtual / ~Bill() |
| OutPatientBill | OutPatientBill::total() | OutPatientBill::type() / ~OutPatientBill() |

| Class | total() | type() / Destructor |
|---|---|---|
| InPatientBill | InPatientBill::total() | InPatientBill::type() / ~InPatientBill() |

When the statement `b->total()` is executed, the program follows the object's vptr to its vtable and calls the correct derived-class function.

### A2.3 – Virtual Destructors

A virtual destructor is required in a polymorphic base class when derived objects may be deleted using base-class pointers.

```
Bill* b = new OutPatientBill();
delete b;
```

If the base destructor is virtual, the derived destructor is called first, followed by the base destructor. If the base destructor is not virtual, deleting a derived object through a base-class pointer causes undefined behaviour.

***Predict the Output***

**Program: Task02/A2_Binding.cpp (Part 1 is the given program)**

```
#include <iostream>
using namespace std;

// ---- Part 1: Predict-the-output program (late binding + virtual destructor) ----
struct Bill {
    virtual ~Bill() { cout << "~Bill "; }
    virtual double total() const = 0;
    void show() const { cout << total() << ' '; }
};

struct OPBill : Bill {
    ~OPBill() override { cout << "~OPBill "; }
    double total() const override { return 500; }
};

// ---------- Part 2: Early binding vs late binding ----------
struct Base {
    void early() const { cout << "Base::early (decided at compile time)" << endl; }
    virtual void late() const { cout << "Base::late" << endl; }
    virtual ~Base() {}
};
```

```
struct Derived : Base {
    void early() const { cout << "Derived::early" << endl; }
    void late() const override {
        cout << "Derived::late (decided at run time)" << endl;
    }
};

// ---------- Part 3: Size of an object with and without a vtable pointer ----------
struct Plain  { int x; void f() {} };
struct Poly   { int x; virtual void f() {} virtual ~Poly() {} };


int main() {
    cout << "=== Part 1: Predict the output ===" << endl;
    Bill* b = new OPBill;
    b->show();
    delete b;
    cout << '\n';


    cout << "\n=== Part 2: Early vs late binding ===" << endl;
    Derived d;
    Base* p = &d;
    p->early();   // non-virtual : uses pointer type  -> Base::early
    p->late();    // virtual     : uses object type   -> Derived::late

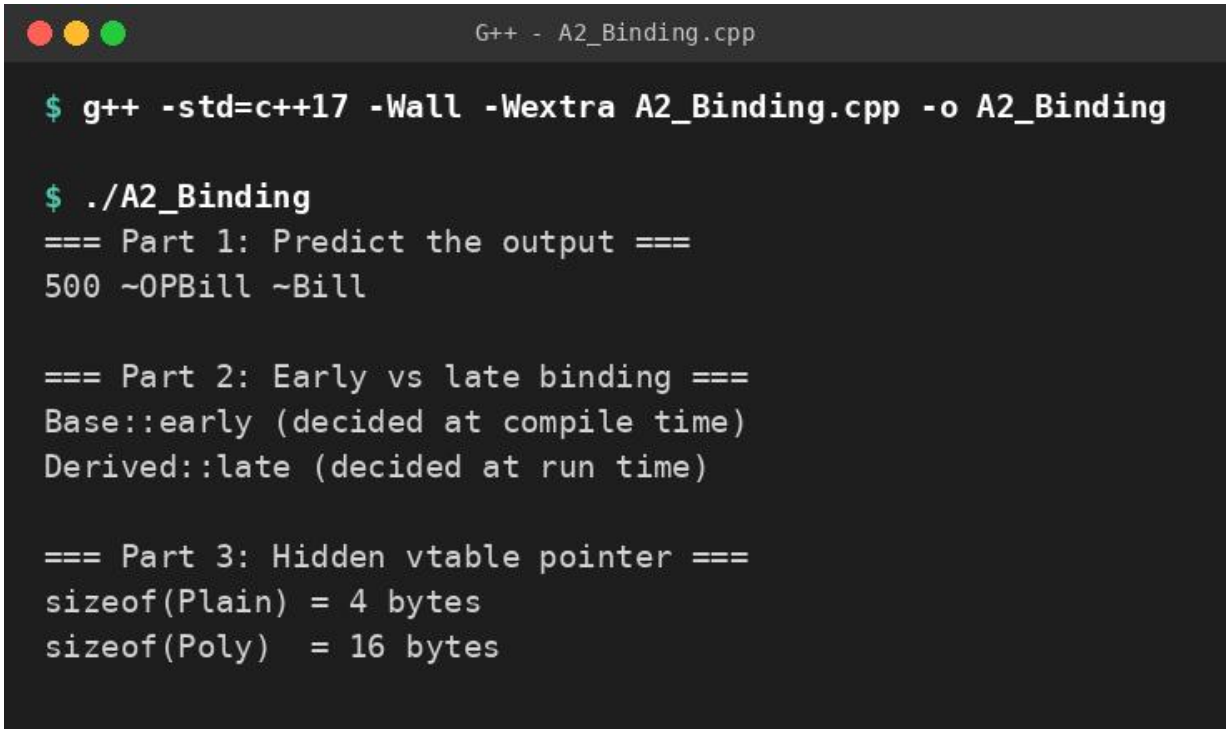

    cout << "\n=== Part 3: Hidden vtable pointer ===" << endl;
    cout << "sizeof(Plain) = " << sizeof(Plain) << " bytes" << endl;
    cout << "sizeof(Poly)  = " << sizeof(Poly)  << " bytes" << endl;
    return 0;
}
```

## My Predicted Output (written before execution):

```
500 ~OPBill ~Bill
```

## Explanation:

1. `b` points to an `OPBill` object.
2. `show()` is non-virtual, but it calls the virtual function `total()`.
3. Runtime binding executes `OPBill::total()`, returning 500.
4. When `delete b` executes, the derived destructor runs first.
5. The base destructor runs next because it is virtual.




*Output Screenshot: A2_Binding.cpp – Part 1 matches the prediction*

Part 2 shows early binding (`p->early()` calls `Base::early`, chosen from the pointer type) and late binding (`p->late()` calls `Derived::late`, chosen from the object type). Part 3 shows the cost of the hidden vptr: the class with virtual functions is larger than the plain class (implementation-dependent; 16 bytes against 4 bytes on this system).

**If the `virtual` keyword is removed from the base destructor:** deleting the derived object through a base pointer causes undefined behaviour and the derived destructor is not guaranteed to run. This was tested with AddressSanitizer in Task B1.4; the actual report is shown there.

### *Alternative Concept and Complexity Analysis*

If `total()` is not virtual, `dynamic_cast` can be used to identify the actual derived type and call its function.

- Virtual approach: one virtual function call per bill.
- `dynamic_cast` approach: may require checking multiple derived types.

- For n bills, virtual processing takes O(n) time.
- If there are k bill types and every bill is checked against them, the cast-based approach can take O(nk) time in the worst case.
- Polymorphic objects commonly contain a hidden vtable pointer, adding implementation-dependent space overhead.

**Conclusion:** Virtual functions provide a simpler and more maintainable way to process different bill types.

## TASK A3 – DESIGN WITH ABSTRACT CLASSES

### A3.1 – Payer Interface

The hospital billing system supports different payment methods. An abstract class called `Payer` defines a common interface.

- **SelfPay** – the patient pays the complete bill amount.
- **Insurance** – the insurance company pays according to the cover limit and the co-payment percentage: `min(total, limit) × (1 − copay)`.
- **GovernmentScheme** – payment is calculated using fixed package rates for each bill type.

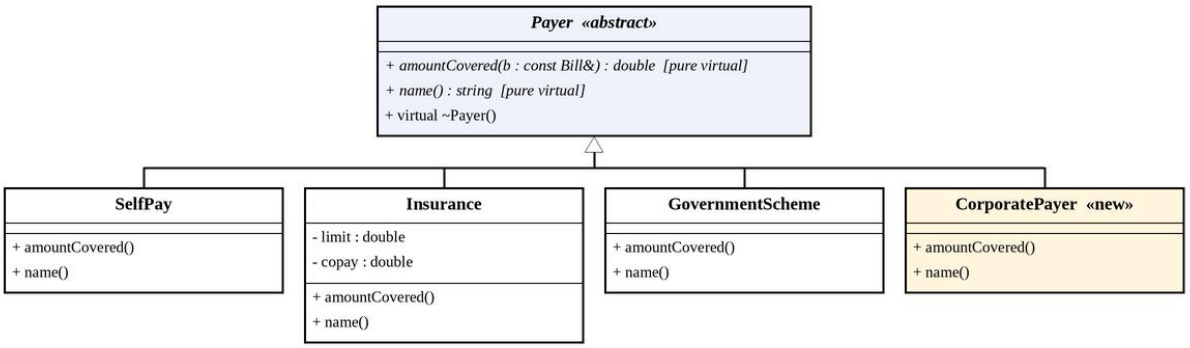



*Figure A3: UML class diagram of the Payer hierarchy*

### Program: Task03/A3_Payer.cpp

```
#include <iostream>
#include <iomanip>
#include <memory>
```

```
#include <string>
#include <vector>
#include <algorithm>
#include <stdexcept>
using namespace std;

// ---------------- Bill hierarchy ----------------
class Bill {
protected:
    string patient;
    int number;

public:
    Bill(const string& p, int n) : patient(p), number(n) {}
    virtual ~Bill() {}

    virtual double total() const = 0;
    virtual string type() const = 0;

    void print() const {
        cout << "Bill #" << number << " [" << type() << "] "
             << patient << " : " << total() << endl;
    }
};

class OutPatientBill : public Bill {
    double consultation, tests;
public:
    OutPatientBill(const string& p, int n, double c, double t)
        : Bill(p, n), consultation(c), tests(t) {}
    double total() const override { return consultation + tests; }
    string type() const override { return "OutPatient"; }
};

class InPatientBill : public Bill {
    int days;
    double roomRate, procedures, medicines;
public:
    InPatientBill(const string& p, int n, int d, double r, double pr, double m)
        : Bill(p, n), days(d), roomRate(r), procedures(pr), medicines(m) {}
    double total() const override {
        return days * roomRate + procedures + medicines;
    }
    string type() const override { return "InPatient"; }
};

class DayCareBill : public Bill {
    double packageAmount;
public:
    DayCareBill(const string& p, int n, double pk)
        : Bill(p, n), packageAmount(pk) {}
    double total() const override { return packageAmount; }
    string type() const override { return "DayCare"; }
};

class EmergencyBill : public Bill {
    double baseCharge, hours, ratePerHour;
public:
    EmergencyBill(const string& p, int n, double b, double h, double r)
        : Bill(p, n), baseCharge(b), hours(h), ratePerHour(r) {}
    double total() const override { return baseCharge + hours * ratePerHour; }
    string type() const override { return "Emergency"; }
};
```

```
// ---------------- Payer hierarchy (abstract interface) ----------------
class Payer {
public:
    virtual double amountCovered(const Bill& b) const = 0;
    virtual string name() const = 0;
    virtual ~Payer() {}
};

class SelfPay : public Payer {
public:
    double amountCovered(const Bill& b) const override { return b.total(); }
    string name() const override { return "SelfPay"; }
};

class Insurance : public Payer {
    double limit;   // maximum amount the insurer will consider
    double copay;   // fraction paid by patient (0.10 = 10 %)
public:
    Insurance(double l, double c) : limit(l), copay(c) {}
    double amountCovered(const Bill& b) const override {
        return min(b.total(), limit) * (1.0 - copay);
    }
    string name() const override { return "Insurance"; }
};

class GovernmentScheme : public Payer {
    static double packageRate(const string& type) {
        if (type == "OutPatient") return 1500;
        if (type == "InPatient")  return 50000;
        if (type == "DayCare")    return 15000;
        if (type == "Emergency")  return 20000;
        throw runtime_error("no package rate for type " + type);
    }
public:
    double amountCovered(const Bill& b) const override {
        return min(b.total(), packageRate(b.type()));
    }
    string name() const override { return "GovtScheme"; }
};

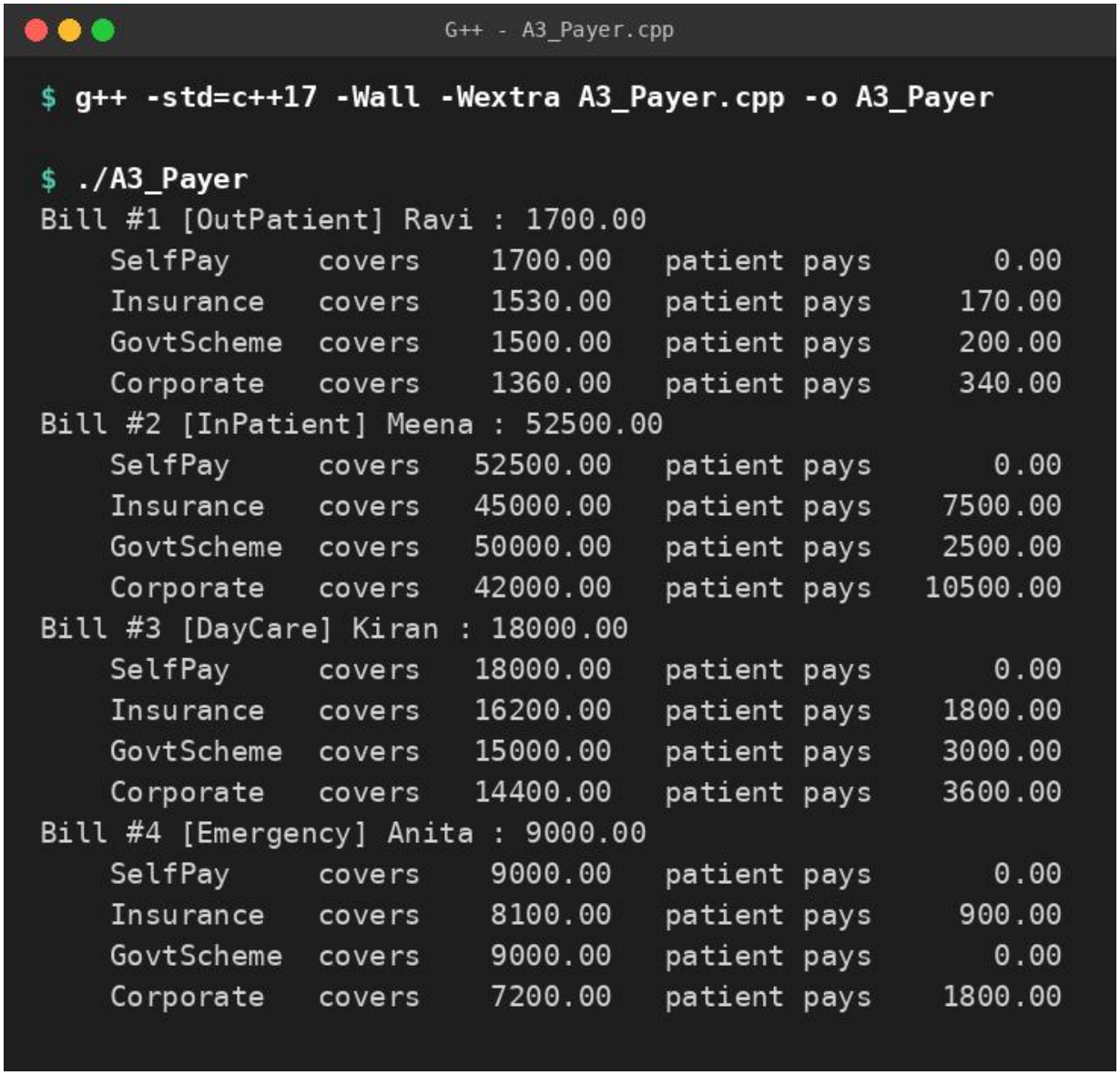
// ---------------- Extension: new payer, no existing class changed ----------------
class CorporatePayer : public Payer {
public:
    double amountCovered(const Bill& b) const override { return b.total() * 0.80; }
    string name() const override { return "Corporate"; }
};

int main() {
    vector<unique_ptr<Bill>> bills;
    bills.push_back(make_unique<OutPatientBill>("Ravi", 1, 500, 1200));
    bills.push_back(make_unique<InPatientBill>("Meena", 2, 3, 2500, 40000, 5000));
    bills.push_back(make_unique<DayCareBill>("Kiran", 3, 18000));
    bills.push_back(make_unique<EmergencyBill>("Anita", 4, 3000, 4, 1500));

    vector<unique_ptr<Payer>> payers;
    payers.push_back(make_unique<SelfPay>());
    payers.push_back(make_unique<Insurance>(50000, 0.10));
    payers.push_back(make_unique<GovernmentScheme>());
    payers.push_back(make_unique<CorporatePayer>());

    cout << fixed << setprecision(2);
    for (const auto& b : bills) {
```

```
        b->print();
        for (const auto& p : payers) {
            double covered = p->amountCovered(*b);
            cout << "    " << left << setw(11) << p->name()
                 << " covers " << right << setw(10) << covered
                 << "   patient pays " << setw(10) << b->total() - covered << endl;
        }
    }
    return 0;
}
```




*Output Screenshot: A3_Payer.cpp*

**Checking one result by hand (Bill #2, in-patient, total = 3 × 2500 + 40000 + 5000 = 52500):** Insurance covers min(52500, 50000) × 0.90 = 45000; the Government

scheme covers min(52500, 50000) = 50000; the Corporate payer covers 52500 × 0.80 = 42000. These match the program output.

### A3.2 – Extension

A new payer, `CorporatePayer`, is added by deriving it from the abstract `Payer` class (it is included in the program above). It covers 80% of the bill amount, and the existing payer classes are not modified.

```
class CorporatePayer : public Payer {
public:
    double amountCovered(const Bill& b) const override {
        return b.total() * 0.80;
    }
    string name() const override { return "Corporate"; }
};
```

### *Alternative Concept and Complexity Analysis*

Instead of an abstract `Payer` class, function pointers or `std::function` objects can store different payment policies.

```
#include <functional>

std::function<double(double)> policy;

policy = [](double amount) {
    return amount * 0.80;
};
```

| **Feature** | **Abstract Payer** | **Function pointer / std::function** |
|---|---|---|
| Design | Class hierarchy | Stored callable policy |
| Runtime call | O(1) | O(1) |
| Data storage | Members in payer objects | Captured data in callable |
| Extension | Add derived class | Add a new callable |
| Reusability | Supports class-based behaviour | Supports flexible policy functions |

**Conclusion:** The abstract payer class is suitable when payment methods have their own data and behaviour. Function objects are useful for simpler and configurable payment rules.

## TASK A4 – DESIGN THE EXCEPTION STRATEGY

### A4.1 – Identify the Errors

| Error | Reason | Exception used |
|---|---|---|
| Negative charges | Bill amount is less than zero | `std::invalid_argument` |
| Claim above cover | Claim exceeds insurance limit | `ClaimError` |
| Unknown package | Invalid government package code | `PackageError` |
| Missing patient record | Patient information is unavailable | `PatientError` |
| Memory allocation failure | Insufficient memory | `std::bad_alloc` |

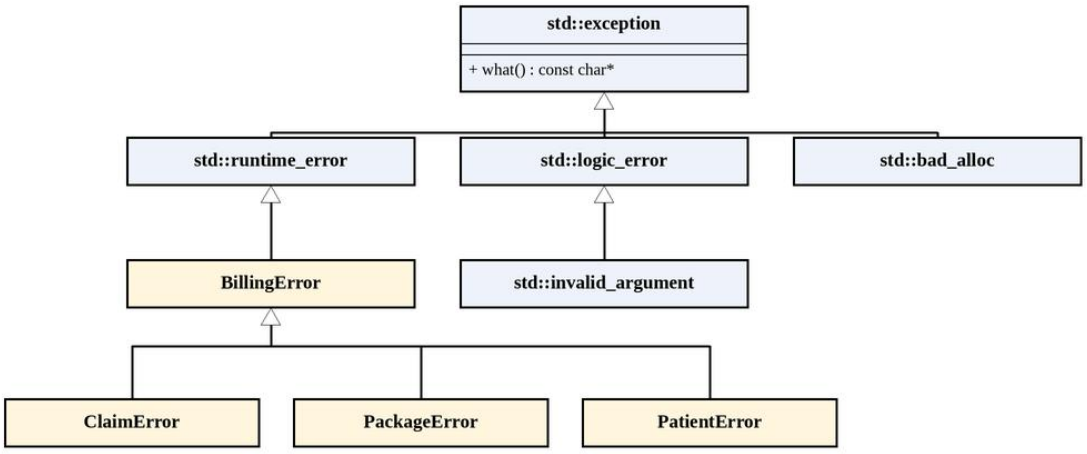



*Figure A4: Exception hierarchy (project classes shaded)*

`std::bad_alloc` is a standard exception used for memory allocation failures.

## Program: Task04/A4_Exception.cpp

```
#include <iostream>
#include <map>
#include <string>
#include <stdexcept>
#include <functional>
#include <limits>
#include <new>
using namespace std;

// ---------------- Exception hierarchy ----------------
class BillingError : public runtime_error {
public:
    using runtime_error::runtime_error;
};
class ClaimError : public BillingError {
public:
    using BillingError::BillingError;
};
class PackageError : public BillingError {
public:
    using BillingError::BillingError;
};
class PatientError : public BillingError {
public:
    using BillingError::BillingError;
};

// ---------------- Functions that may throw ----------------
void validateAmount(double amount) {
    if (amount < 0) throw invalid_argument("negative charge: " + to_string(amount));
}

double approveClaim(double claim, double cover) {
    if (claim > cover) throw ClaimError("claim " + to_string((long)claim) +
                                        " exceeds cover " + to_string((long)cover));
    return claim;
}

double findPackage(const string& code) {
    static const map<string, double> packages = {
        {"PKG-CAT", 15000}, {"PKG-APP", 30000}};
    auto it = packages.find(code);
    if (it == packages.end()) throw PackageError("unknown package code " + code);
    return it->second;
}

string findPatient(int id) {
    static const map<int, string> records = {{101, "Ravi"}, {102, "Meena"}};
    auto it = records.find(id);
    if (it == records.end())
        throw PatientError("missing patient record " + to_string(id));
    return it->second;
}

void allocateTooMuch() {
    volatile size_t n = numeric_limits<size_t>::max() / 4;   // impossible request
    char* p = new char[n];
    delete[] p;
```

```
}

// ---------------- Test driver ----------------
void run(const string& label, const function<void()>& test) {
    cout << label << " -> ";
    try {
        test();
        cout << "no exception" << endl;
    }
    catch (const ClaimError& e) {
        cout << "ClaimError: " << e.what() << endl;
    }
    catch (const PackageError& e) {
        cout << "PackageError: " << e.what() << endl;
    }
    catch (const PatientError& e) {
        cout << "PatientError: " << e.what() << endl;
    }
    catch (const BillingError& e) {
        cout << "BillingError: " << e.what() << endl;
    }
    catch (const invalid_argument& e) {
        cout << "invalid_argument: " << e.what() << endl;
    }
    catch (const bad_alloc& e) {
        cout << "bad_alloc: " << e.what() << endl;
    }
    catch (const exception& e) {
        cout << "std::exception: " << e.what() << endl;
    }
    catch (...) {
        cout << "unknown exception" << endl;
    }
}

int main() {
    run("Valid charge 1700          ", [] { validateAmount(1700); });
    run("Negative charge -500       ", [] { validateAmount(-500); });
    run("Claim 60000 vs cover 50000 ", [] { approveClaim(60000, 50000); });
    run("Unknown package PKG-XYZ    ", [] { findPackage("PKG-XYZ"); });
    run("Missing patient id 999     ", [] { findPatient(999); });
    run("Text 'abc' using stod      ", [] { stod("abc"); });
    run("Impossible allocation      ", [] { allocateTooMuch(); });
    return 0;
}
```

```
G++ - A4_Exception.cpp

$ g++ -std=c++17 -Wall -Wextra A4_Exception.cpp -o A4_Exception

$ ./A4_Exception
Valid charge 1700           -> no exception
Negative charge -500        -> invalid_argument: negative charge: -500.000000
Claim 60000 vs cover 50000  -> ClaimError: claim 60000 exceeds cover 50000
Unknown package PKG-XYZ     -> PackageError: unknown package code PKG-XYZ
Missing patient id 999      -> PatientError: missing patient record 999
Text 'abc' using stod       -> invalid_argument: stod
Impossible allocation       -> bad_alloc: std::bad_alloc
```

*Output Screenshot: A4_Exception.cpp*

### A4.2 – Specify Exceptions

Functions that validate charges, process claims or search for patient records may throw exceptions.

- `validateAmount()` may throw `invalid_argument`.
- `approveClaim()` may throw `ClaimError`.
- `findPackage()` may throw `PackageError`.
- `findPatient()` may throw `PatientError`.
- Memory allocation may throw `bad_alloc`.

A destructor should not throw exceptions. Throwing from a destructor during stack unwinding can terminate the program. Functions that must not throw should be marked `noexcept` only when their operations genuinely cannot throw.

#### *Alternative Concept and Complexity Analysis*

Errors can also be reported using return codes instead of exceptions.

```
enum Status {
    SUCCESS,
    INVALID_AMOUNT,
    CLAIM_EXCEEDED
};
```

| Feature | Exceptions | Return Codes |
|---|---|---|
| Normal execution | No error-handling branch required at every call | Caller checks status |
| Error handling | Uses try and catch | Uses if and switch |
| Time | Usually low overhead on normal path | Explicit checks on each path |
| Extra data | Exception object when thrown | Status and possibly message |
| Code | Centralised handling | Error checks across functions |

**Conclusion:** Exceptions are useful for separating normal program logic from error-handling logic. Return codes may be suitable when errors occur frequently and predictable handling is needed (measured in Task B3.3).

## TASK A5 – DESIGN RECOVERY AND RETHROWING

### A5.1 – Where to Catch

Exception handling should be performed at the appropriate level.

1. Low-level functions detect errors and throw exceptions.
2. Intermediate functions may catch exceptions for logging.
3. They can rethrow the same exception using `throw;`.
4. The `main` function catches the exception and displays a meaningful message to the user.

### A5.2 – No Leaks on Errors

Stack unwinding is the process in which local objects are destroyed when an exception leaves a function. RAII (Resource Acquisition Is Initialization) ensures that resources are released automatically through destructors.

- `std::vector` manages dynamic arrays.
- `std::unique_ptr` manages dynamically allocated objects.
- File stream objects close files automatically.

RAII prevents resource leaks even when exceptions occur.

***Predict the Output***

**Program: Task05/A5_Recovery.cpp (Part 1 is the given program)**

```
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>
using namespace std;


// ---------------- Part 1: Predict-the-output program ----------------
```

```
struct Guard {
    string n;
    explicit Guard(const string& s) : n(s) {}
    ~Guard() { cout << '~' << n << ' '; }
};

void f() {
    Guard g("f");
    throw 42;
}

void part1() {
    try {
        Guard m("main");
        f();
        cout << "not printed ";
    }
    catch (int x) {
        cout << "caught " << x;
    }
    catch (...) {
        cout << "other";
    }
    cout << '\n';
}

// ---------------- Part 2: where to catch / rethrow ----------------
class ClaimError : public runtime_error {
public:
    using runtime_error::runtime_error;
};

// lowLevel detects the error and throws
void lowLevel(double claim, double cover) {
    if (claim > cover) throw ClaimError("claim exceeds cover");
}

// midLevel logs, then rethrows the SAME exception
void midLevel(double claim, double cover) {
    try {
        lowLevel(claim, cover);
    }
    catch (const ClaimError& e) {
        cout << "[log] midLevel saw: " << e.what() << endl;
        throw;
    }
}

void part2() {
    try {
        midLevel(60000, 50000);
    }
    catch (const exception& e) {   // top level shows a user message
        cout << "Error: " << e.what() << endl;
    }
}

// ---------------- Part 3: RAII - no leaks on errors ----------------
struct Resource {
    string name;
    explicit Resource(const string& n) : name(n) {
        cout << "acquire " << name << endl;
    }
```

```
    ~Resource() { cout << "release " << name << endl; }
};

void riskyWork() {
    auto r1 = make_unique<Resource>("bill-buffer");    // RAII owners
    auto r2 = make_unique<Resource>("claim-file");
    throw runtime_error("failure while processing");   // both are still released
}

void part3() {
    try { riskyWork(); }
    catch (const exception& e) { cout << "caught: " << e.what() << endl; }
}

int main() {
    cout << "--- Part 1 ---" << endl;  part1();
    cout << "--- Part 2 ---" << endl;  part2();
    cout << "--- Part 3 ---" << endl;  part3();
    return 0;
}
```

**My Predicted Output for Part 1 (written before execution):**

```
~f ~main caught 42
```

**Explanation:**

1. The `Guard` object `m` is created in `main()`.
2. The function `f()` creates another object named `g`.
3. The statement `throw 42` throws an integer exception.
4. The destructor of `g` executes first.
5. The destructor of `m` executes next during stack unwinding.
6. The integer exception is caught by `catch (int x)`.
7. The message `caught 42` is printed.
8. `not printed` does not appear because the exception transfers control before that statement.




*Output Screenshot: A5_Recovery.cpp – Part 1 matches the prediction; Parts 2 and 3 show rethrowing and RAII*

In Part 3 both resources are released in reverse order of acquisition even though `riskyWork()` exits through an exception.

### *Alternative Concept and Complexity Analysis*

If memory is released manually using `delete` in every catch block, every error path must be handled carefully. With RAII, destructors release resources automatically.

| **Feature** | **RAII** | **Manual delete** |
|---|---|---|
| Resource release | Automatic | Programmer-controlled |
| Time complexity | O(1) per resource release | O(1) per resource release |
| Code size | Less cleanup code | More cleanup code |
| Leak risk | Lower | Higher |
| Error handling | Safer | Easy to miss a cleanup path |

**Conclusion:** RAII is a reliable method for resource management because it automatically releases resources when objects go out of scope.

## PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION (25 MARKS)

## TASK B1 – IMPLEMENT AND VALIDATE THE POLYMORPHIC BILLING

### B1.1 – Validate the Implementation

The following program implements an abstract `Bill` class and two derived classes, `OutPatient` and `InPatient`. It uses runtime polymorphism to calculate bill amounts.

### Program: Task01/B1_Billing.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Bill {
protected:
    string patient;

public:
    Bill(const string& p) : patient(p) {}

    virtual ~Bill() {}

    virtual double total() const = 0;
    virtual string type() const = 0;

    void print() const {
        cout << type() << " " << patient
             << ": " << total() << endl;
    }
};

class OutPatient : public Bill {
    double consultation, tests;

public:
    OutPatient(const string& p, double c, double t)
        : Bill(p), consultation(c), tests(t) {}

    double total() const override {
        return consultation + tests;
    }

    string type() const override {
        return "OP";
    }
};
```

```
class InPatient : public Bill {
    int days;
    double perDay, surgery;

public:
    InPatient(const string& p, int d, double rate, double s)
        : Bill(p), days(d), perDay(rate), surgery(s) {}

    double total() const override {
        return days * perDay + surgery;
    }

    string type() const override {
        return "IP";
    }
};

int main() {
    Bill* bills[] = {
        new OutPatient("Ravi", 500, 1200),
        new InPatient("Meena", 3, 2500, 40000)
    };

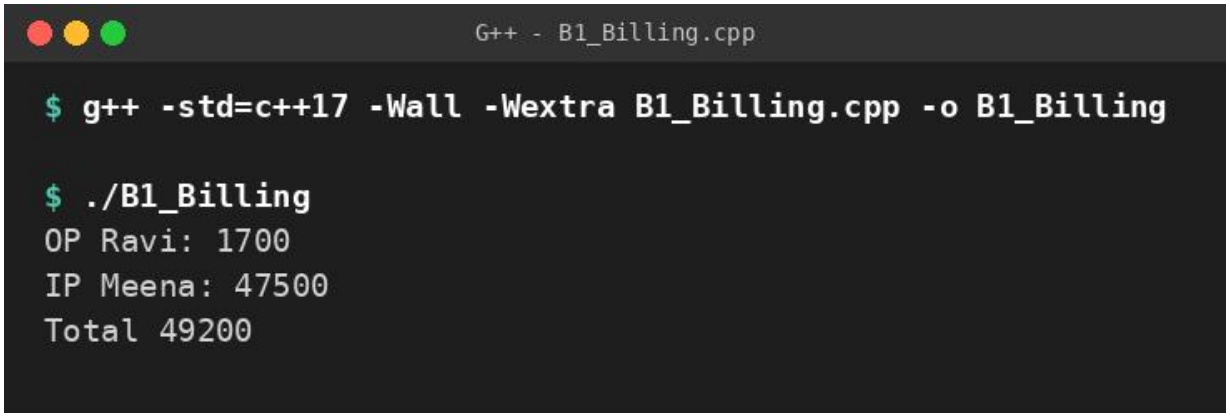
    double sum = 0;

    for (Bill* b : bills) {
        b->print();
        sum += b->total();
    }

    cout << "Total " << sum << endl;

    for (Bill* b : bills) {
        delete b;
    }

    return 0;
}
```




*Output Screenshot: B1_Billing.cpp*

**Verification by hand:**

- OutPatient = 500 + 1200 = 1700
- InPatient = (3 × 2500) + 40000 = 47500
- Total = 1700 + 47500 = 49200

The program output equals the hand calculation.

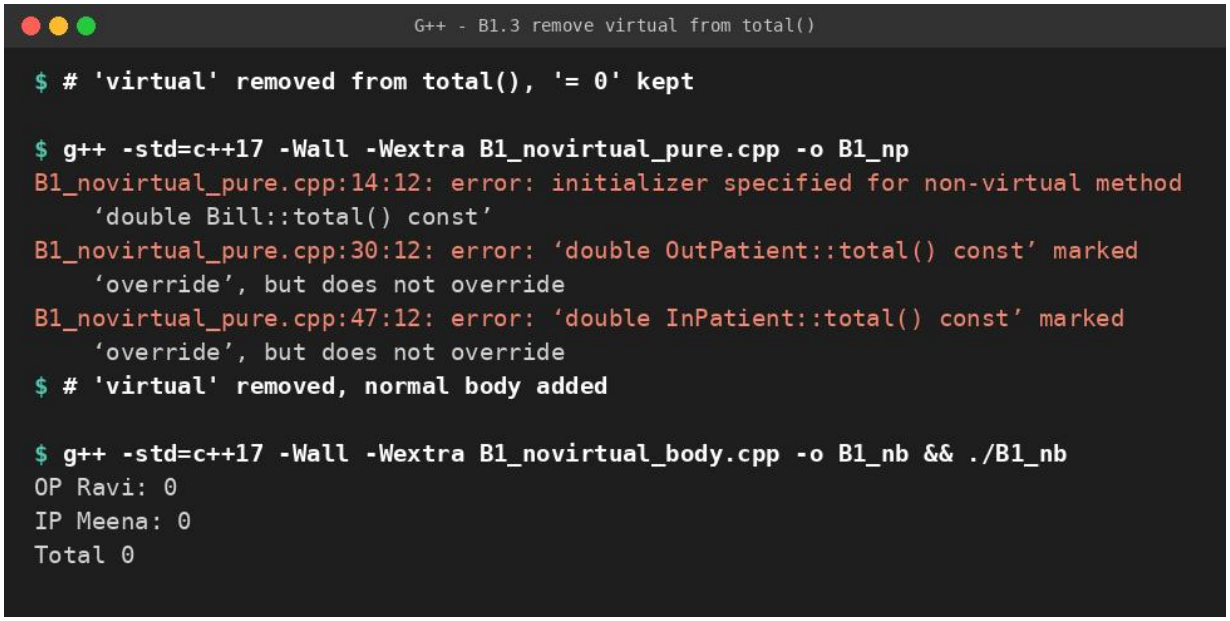
### B1.2 – Compare with Your Design

The implementation follows the design in Part A. The abstract base class `Bill` provides the common interface. `OutPatient` and `InPatient` override the virtual functions `total()` and `type()`. The `print()` function uses the virtual functions to display the correct bill details. A virtual destructor is used to ensure safe deletion of derived objects. The remaining bill types (`DayCareBill`, `EmergencyBill`) and the payer classes are implemented in Task A3 and Task B4.

### B1.3 – Prove Runtime Binding

Two experiments were performed by removing the `virtual` keyword from `Bill::total()`:

1. **Keeping `= 0`:** `double total() const = 0;` is rejected by the compiler, because only virtual functions can be pure. The derived-class `override` specifiers also fail, since there is no virtual function left to override. The program does not compile.
2. **Replacing `= 0` with a normal body (`return 0;`) and removing `override`:** the program compiles, but `b->total()` is now bound at compile time to `Bill::total()` through the base-class pointer. Every bill prints 0 instead of its real amount.



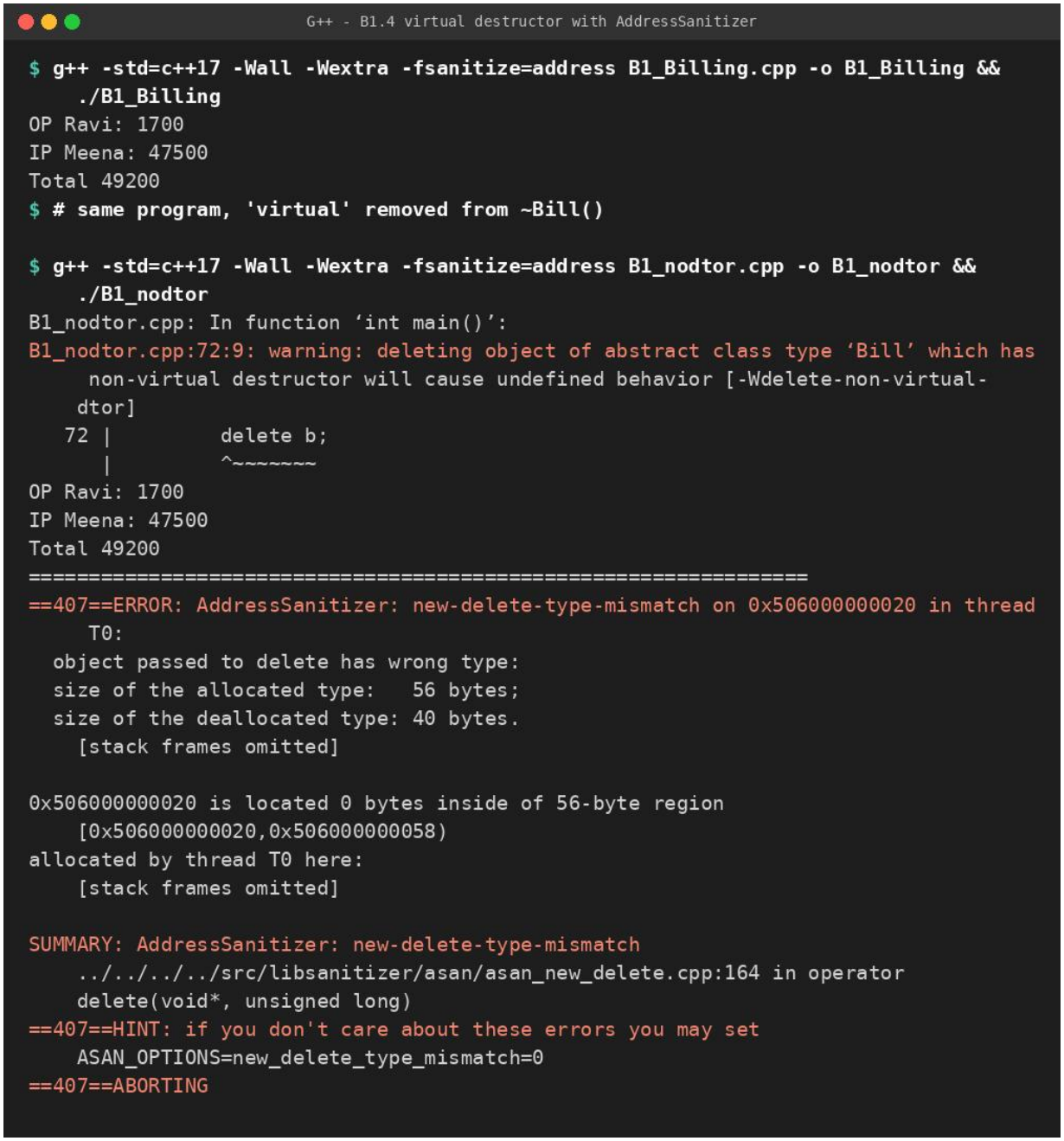

*Output Screenshot: effect of removing virtual from total()*

**Conclusion:** the correct totals (1700, 47500, 49200) are obtained only when `total()` is virtual, which proves that the function is selected at run time from the actual object type.

### B1.4 – Prove the Virtual Destructor

The virtual destructor ensures that the correct derived destructor is called when an object is deleted through a base-class pointer. The program was compiled with AddressSanitizer using:

```
g++ -std=c++17 -Wall -Wextra -fsanitize=address B1_Billing.cpp -o B1_Billing
```




*Output Screenshot: AddressSanitizer – correct program (clean) and the version without the virtual destructor*

**Result:** with the virtual destructor the program runs with no sanitizer report. When `virtual` is removed from `~Bill()`, the compiler warns (`-Wdelete-non-virtual-dtor`) and AddressSanitizer aborts the run with **new-delete-type-mismatch**: the allocated object is 56 bytes (the derived class) but only 40 bytes (the base class) are released. This confirms the undefined behaviour.

*Alternative Concept and Complexity Analysis*

Templates can process each bill type separately without virtual function calls.

- Virtual function approach: all bill types can be stored in one base-class pointer collection.
- Template approach: each bill type can be stored in its own container.
- Both approaches take O(n) time to process n bills.
- Templates avoid virtual function dispatch, but separate containers make it less convenient to maintain one mixed list of all bills.

**Conclusion:** Runtime polymorphism is suitable when different bill types must be processed through one common interface.

## TASK B2 – ENGINEER ERROR CONDITIONS

### B2.1 – Error Testing

The following test cases check the exception-handling mechanism. The actual results come from running `A4_Exception.cpp` (Task A4).

| Test case | Input | Expected behaviour | Actual result |
|---|---|---|---|
| Negative charges | -500 | Throw invalid_argument | invalid_argument |
| Claim above cover | 60000, 50000 | Throw ClaimError | ClaimError |
| Unknown package | PKG-XYZ | Throw package exception | PackageError |
| Text instead of number | "abc" | stod throws invalid_argument | invalid_argument |

| Test case | Input | Expected behaviour | Actual result |
|---|---|---|---|
| Memory allocation failure | Impossibly large request | May throw bad_alloc | bad_alloc |

A `catch (...)` block can be used at the top level to catch exceptions that are not handled by earlier handlers. Local objects are destroyed during stack unwinding.

***Predict the Output***

**Program: Task05/B2_Predict.cpp (given program)**

```
#include <iostream>
#include <stdexcept>

using namespace std;

void validate(double amount) {
    if (amount < 0)
        throw invalid_argument("negative");

    if (amount > 1e6)
        throw out_of_range("too large");

    cout << "ok ";
}

void process(double amount) {
    try {
        validate(amount);
    }
    catch (const out_of_range& e) {
        cout << "[log " << e.what() << "] ";
        throw;
    }
}

int main() {
    for (double a : {100.0, -5.0, 2e6}) {
        try {
            process(a);
            cout << "done ";
        }
        catch (const exception& e) {
            cout << "caught " << e.what() << ' ';
        }
    }

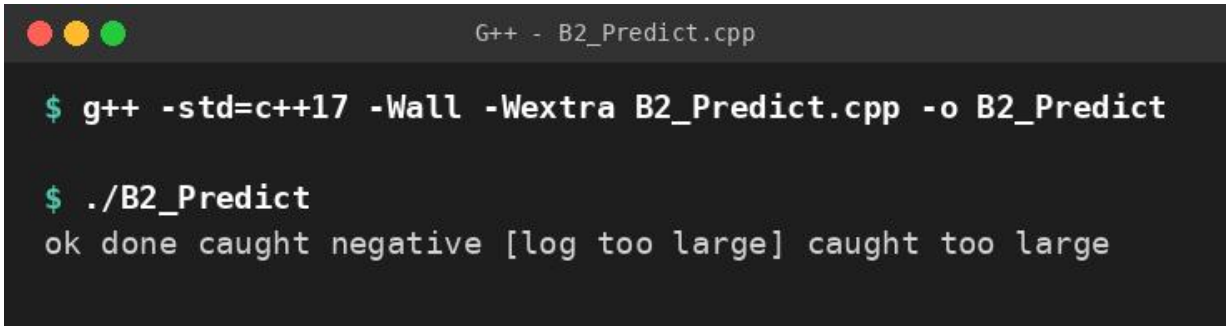
    cout << endl;
    return 0;
}
```

**My Predicted Output (written before execution):**

```
ok done caught negative [log too large] caught too large
```

**Explanation:**

1. For 100.0, validation succeeds, so `ok done` is printed.
2. For -5.0, `invalid_argument("negative")` is thrown. The `out_of_range` handler does not catch it, so it reaches the outer handler.
3. For 2e6, `out_of_range("too large")` is thrown.
4. The inner handler logs the message and rethrows the same exception using `throw;`.
5. The outer handler catches it and prints the error.




*Output Screenshot: B2_Predict.cpp – matches the prediction*

Only the `out_of_range` exception is logged because the inner function catches only that exception type.

**B2.2 – Complete the Code**

**Program: Task05/B2_Claim.cpp**

```
#include <iostream>
#include <stdexcept>
using namespace std;


class ClaimError : public runtime_error {
public:
    using runtime_error::runtime_error;
};
```

```
double approve(double billed, double cover) {
    if (billed < 0 || cover < 0) {
        throw invalid_argument("negative amount");
    }

    if (billed > cover) {
        throw ClaimError("exceeds cover");
    }

    return billed;
}

double settle(double billed, double cover) {
    try {
        return approve(billed, cover);
    }
    catch (const ClaimError& e) {
        cout << "audit: " << e.what() << endl;
        throw;
    }
}

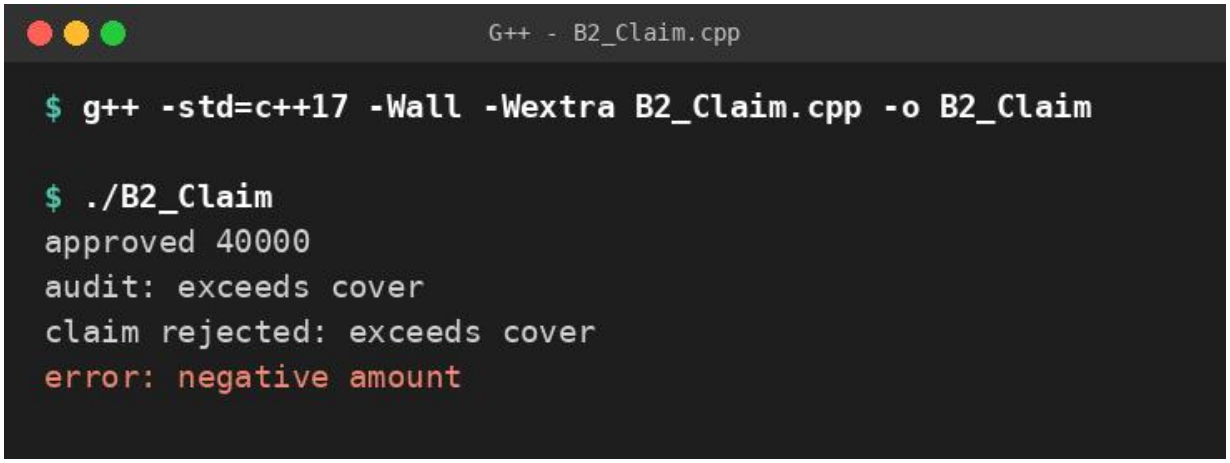
int main() {
    double cases[][2] = {
        {40000, 50000},
        {60000, 50000},
        {-1, 50000}
    };

    for (auto& c : cases) {
        try {
            double paid = settle(c[0], c[1]);
            cout << "approved " << paid << endl;
        }
        catch (const ClaimError& e) {
            cout << "claim rejected: " << e.what() << endl;
        }
        catch (const exception& e) {
            cout << "error: " << e.what() << endl;
        }
    }

    return 0;
}
```

## Expected Output:

```
approved 40000
audit: exceeds cover
claim rejected: exceeds cover
error: negative amount
```




*Output Screenshot: B2_Claim.cpp – matches the expected output*

**Explanation of `throw;` and `throw e;`:**

- `throw;` rethrows the currently handled exception without changing its original type.
- `throw e;` throws a new exception object based on the declared type of `e`. It can cause object slicing when a derived exception is caught through a base-class reference.
- Therefore, `throw;` is preferred when the same exception must be rethrown.

***Alternative Concept and Complexity Analysis***

Using `std::optional<double>`, a function can return a value when the claim is approved or an empty optional when the claim is rejected.

```
#include <optional>

optional<double> approveClaim(double billed, double cover) {
    if (billed < 0 || cover < 0 || billed > cover) {
        return nullopt;
    }

    return billed;
}
```

Both approaches require O(n) time to process n claims. Return codes or optional values are often preferable when invalid claims are frequent. Exceptions are useful

for exceptional error conditions. The measured cost for one million claims is given in Task B3.3.

## TASK B3 – VIRTUAL FUNCTIONS vs SWITCH vs EXCEPTIONS: THE COST

### B3.1 – Correctness on Small Data

The same 10 bills are processed using virtual functions through a base-class pointer, a single `Bill` structure with a `switch` statement, and `std::variant`. All must calculate the same total.

| Bill number | Bill type | Amount |
|---|---|---|
| 1 | OutPatient | 1700 |
| 2 | InPatient | 47500 |
| 3 | OutPatient | 2500 |
| 4 | InPatient | 30000 |
| 5 | OutPatient | 1200 |
| 6 | InPatient | 55000 |
| 7 | OutPatient | 3000 |
| 8 | InPatient | 42000 |
| 9 | OutPatient | 1800 |
| 10 | InPatient | 35000 |

**Total calculation:**

```
1700 + 47500 + 2500 + 30000 + 1200 + 55000 + 3000 + 42000 + 1800 + 35000 = 219700
```

**Result:** all three implementations produce a total of **219700** (see the first line of each program's output in the screenshot below).

### B3.2 – Increase the Data Size

The performance of the approaches was measured using different numbers of bills. The bills are generated with a fixed pseudo-random sequence (half out-patient, half in-patient), so all programs process identical data; each program prints the same sum for each size. The programs were compiled with optimization using `-O2`, and only the processing loop is timed. The best time of 9 passes (5 passes for 10,000,000 bills) is reported.

```
g++ -std=c++17 -O2 -Wall -Wextra B3_Virtual.cpp -o B3_Virtual
g++ -std=c++17 -O2 -Wall -Wextra B3_Switch.cpp  -o B3_Switch
g++ -std=c++17 -O2 -Wall -Wextra B3_Variant.cpp -o B3_Variant
```

### Program: Task06/B3_Virtual.cpp

```
#include <iostream>
#include <vector>
#include <memory>
#include <chrono>
#include <string>
#include <algorithm>
using namespace std;
using Clock = chrono::steady_clock;

class Bill {
public:
    virtual ~Bill() {}
    virtual double total() const = 0;
};

class OutPatient : public Bill {
    double consultation, tests;
public:
    OutPatient(double c, double t) : consultation(c), tests(t) {}
    double total() const override { return consultation + tests; }
};

class InPatient : public Bill {
    int days;
    double perDay, surgery;
public:
    InPatient(int d, double r, double s) : days(d), perDay(r), surgery(s) {}
    double total() const override { return days * perDay + surgery; }
};

static unsigned long long state = 12345;
static unsigned next() {            // simple deterministic random generator
    state = state * 6364136223846793005ULL + 1442695040888963407ULL;
    return (unsigned)(state >> 33);
}

vector<unique_ptr<Bill>> makeBills(size_t n) {
    vector<unique_ptr<Bill>> v;
```

```
    v.reserve(n);
    for (size_t i = 0; i < n; i++) {
        // values are drawn into locals first: the evaluation order of function
        // arguments is unspecified in C++, which would change the generated data
        if (next() % 2 == 0) {
            double c = 500 + next() % 1000;
            double t = 200 + next() % 2000;
            v.push_back(make_unique<OutPatient>(c, t));
        } else {
            int    d = 1 + next() % 7;
            double r = 2000 + next() % 1500;
            double s = 10000 + next() % 40000;
            v.push_back(make_unique<InPatient>(d, r, s));
        }
    }
    return v;
}

double processAll(const vector<unique_ptr<Bill>>& v) {
    double sum = 0;
    for (const auto& b : v) sum += b->total();
    return sum;
}

int main() {
    // B3.1 correctness on the 10 sample bills
    vector<unique_ptr<Bill>> small;
    small.push_back(make_unique<OutPatient>(500, 1200));      // 1700
    small.push_back(make_unique<InPatient>(3, 2500, 40000));  // 47500
    small.push_back(make_unique<OutPatient>(1000, 1500));     // 2500
    small.push_back(make_unique<InPatient>(4, 2500, 20000));  // 30000
    small.push_back(make_unique<OutPatient>(400, 800));       // 1200
    small.push_back(make_unique<InPatient>(5, 3000, 40000));  // 55000
    small.push_back(make_unique<OutPatient>(1000, 2000));     // 3000
    small.push_back(make_unique<InPatient>(4, 3000, 30000));  // 42000
    small.push_back(make_unique<OutPatient>(600, 1200));      // 1800
    small.push_back(make_unique<InPatient>(2, 2500, 30000));  // 35000
    cout << "Virtual: total of 10 sample bills = " << processAll(small) << endl;


    // B3.2 timing (best of several passes, processing only)
    cout << "Virtual function timing" << endl;
    for (size_t n : {1000UL, 1000000UL, 10000000UL}) {
        auto bills = makeBills(n);
        int reps = (n <= 1000000) ? 9 : 5;
        double best = 1e30, sum = 0;
        for (int r = 0; r < reps; r++) {
            auto t0 = Clock::now();
            sum = processAll(bills);
            auto t1 = Clock::now();
            best = min(best, chrono::duration<double, milli>(t1 - t0).count());
        }
        cout << "n = " << n << "  time = " << best
             << " ms  (sum = " << sum << ")" << endl;
    }
    return 0;
}
```

## Program: Task06/B3_Switch.cpp

```
#include <iostream>
#include <vector>
#include <chrono>
#include <algorithm>
```

```
using namespace std;
using Clock = chrono::steady_clock;


enum BillType { OUTPATIENT, INPATIENT };


struct Bill {
    BillType type;
    double consultation, tests;          // used by OUTPATIENT
    int days; double perDay, surgery;    // used by INPATIENT
};


double total(const Bill& b) {
    switch (b.type) {
        case OUTPATIENT: return b.consultation + b.tests;
        case INPATIENT:  return b.days * b.perDay + b.surgery;
    }
    return 0;
}


static unsigned long long state = 12345;
static unsigned next() {            // simple deterministic random generator
    state = state * 6364136223846793005ULL + 1442695040888963407ULL;
    return (unsigned)(state >> 33);
}


vector<Bill> makeBills(size_t n) {
    vector<Bill> v;
    v.reserve(n);
    for (size_t i = 0; i < n; i++) {
        Bill b{};
        if (next() % 2 == 0) {
            b.type = OUTPATIENT;
            b.consultation = 500 + next() % 1000;
            b.tests = 200 + next() % 2000;
        } else {
            b.type = INPATIENT;
            b.days = 1 + next() % 7;
            b.perDay = 2000 + next() % 1500;
            b.surgery = 10000 + next() % 40000;
        }
        v.push_back(b);
    }
    return v;
}


double processAll(const vector<Bill>& v) {
    double sum = 0;
    for (const auto& b : v) sum += total(b);
    return sum;
}


int main() {
    vector<Bill> small = {
        {OUTPATIENT, 500, 1200, 0, 0, 0},   {INPATIENT, 0, 0, 3, 2500, 40000},
        {OUTPATIENT, 1000, 1500, 0, 0, 0},  {INPATIENT, 0, 0, 4, 2500, 20000},
        {OUTPATIENT, 400, 800, 0, 0, 0},    {INPATIENT, 0, 0, 5, 3000, 40000},
        {OUTPATIENT, 1000, 2000, 0, 0, 0},  {INPATIENT, 0, 0, 4, 3000, 30000},
        {OUTPATIENT, 600, 1200, 0, 0, 0},   {INPATIENT, 0, 0, 2, 2500, 30000}
    };
    cout << "Switch: total of 10 sample bills = " << processAll(small) << endl;


    cout << "Switch statement timing" << endl;
    for (size_t n : {1000UL, 1000000UL, 10000000UL}) {
```

```
        auto bills = makeBills(n);
        int reps = (n <= 1000000) ? 9 : 5;
        double best = 1e30, sum = 0;
        for (int r = 0; r < reps; r++) {
            auto t0 = Clock::now();
            sum = processAll(bills);
            auto t1 = Clock::now();
            best = min(best, chrono::duration<double, milli>(t1 - t0).count());
        }
        cout << "n = " << n << "  time = " << best
             << " ms  (sum = " << sum << ")" << endl;
    }
    return 0;
}
```

## Program: Task06/B3_Variant.cpp

```
#include <iostream>
#include <vector>
#include <variant>
#include <chrono>
#include <algorithm>
using namespace std;
using Clock = chrono::steady_clock;


struct OutPatientBill {
    double consultation, tests;
    double total() const { return consultation + tests; }
};


struct InPatientBill {
    int days; double rate, surgery;
    double total() const { return days * rate + surgery; }
};


using AnyBill = variant<OutPatientBill, InPatientBill>;


static unsigned long long state = 12345;
static unsigned next() {            // simple deterministic random generator
    state = state * 6364136223846793005ULL + 1442695040888963407ULL;
    return (unsigned)(state >> 33);
}


vector<AnyBill> makeBills(size_t n) {
    vector<AnyBill> v;
    v.reserve(n);
    for (size_t i = 0; i < n; i++) {
        if (next() % 2 == 0) {
            double c = 500 + next() % 1000;
            double t = 200 + next() % 2000;
            v.push_back(OutPatientBill{c, t});
        } else {
            int    d = 1 + next() % 7;
            double r = 2000 + next() % 1500;
            double s = 10000 + next() % 40000;
            v.push_back(InPatientBill{d, r, s});
        }
    }
    return v;
}
```

```
double processAll(const vector<AnyBill>& v) {
    double sum = 0;
    for (const auto& b : v)
        sum += visit([](const auto& x) { return x.total(); }, b);
    return sum;
}

int main() {
    vector<AnyBill> small = {
        OutPatientBill{500, 1200},  InPatientBill{3, 2500, 40000},
        OutPatientBill{1000, 1500}, InPatientBill{4, 2500, 20000},
        OutPatientBill{400, 800},   InPatientBill{5, 3000, 40000},
        OutPatientBill{1000, 2000}, InPatientBill{4, 3000, 30000},
        OutPatientBill{600, 1200},  InPatientBill{2, 2500, 30000}
    };
    cout << "Variant: total of 10 sample bills = " << processAll(small) << endl;

    cout << "std::variant timing" << endl;
    for (size_t n : {1000UL, 1000000UL, 10000000UL}) {
        auto bills = makeBills(n);
        int reps = (n <= 1000000) ? 9 : 5;
        double best = 1e30, sum = 0;
        for (int r = 0; r < reps; r++) {
            auto t0 = Clock::now();
            sum = processAll(bills);
            auto t1 = Clock::now();
            best = min(best, chrono::duration<double, milli>(t1 - t0).count());
        }
        cout << "n = " << n << "  time = " << best
             << " ms  (sum = " << sum << ")" << endl;
    }
    return 0;
}
```

```
G++ - B3 virtual vs switch vs variant (-O2)

$ g++ -std=c++17 -O2 -Wall -Wextra B3_Virtual.cpp -o B3_Virtual && ./B3_Virtual
Virtual: total of 10 sample bills = 219700
Virtual function timing
n = 1000  time = 0.00578 ms  (sum = 2.14922e+07)
n = 1000000  time = 9.1691 ms  (sum = 2.16109e+10)
n = 10000000  time = 90.0142 ms  (sum = 2.15802e+11)
$ g++ -std=c++17 -O2 -Wall -Wextra B3_Switch.cpp -o B3_Switch && ./B3_Switch
Switch: total of 10 sample bills = 219700
Switch statement timing
n = 1000  time = 0.000761 ms  (sum = 2.14922e+07)
n = 1000000  time = 7.40914 ms  (sum = 2.16109e+10)
n = 10000000  time = 82.5616 ms  (sum = 2.15802e+11)
$ g++ -std=c++17 -O2 -Wall -Wextra B3_Variant.cpp -o B3_Variant && ./B3_Variant
Variant: total of 10 sample bills = 219700
std::variant timing
n = 1000  time = 0.000743 ms  (sum = 2.14922e+07)
n = 1000000  time = 5.61211 ms  (sum = 2.16109e+10)
n = 10000000  time = 78.2727 ms  (sum = 2.15802e+11)
```

*Output Screenshot: B3 timing results (g++ -O2)*

**Experimental Results Table (time in milliseconds):**

| Number of bills | Virtual function time (ms) | Switch time (ms) | std::variant time (ms) |
|---|---|---|---|
| 1,000 | 0.00578 | 0.00076 | 0.00074 |
| 1,000,000 | 9.17 | 7.41 | 5.61 |
| 10,000,000 | 90.01 | 82.56 | 78.27 |

*Measured on Linux (x86-64) with g++ 13.3.0, `-std=c++17 -O2`. Absolute times depend on the machine, so they will differ on other computers.*

**Observations:**

- Time grows roughly in proportion to the number of bills for every approach (about 9.8 times more time for 10 times more bills with virtual functions), which agrees with the O(n) analysis.
- For 10,000,000 bills the virtual-function version took 90.01 ms and the switch version 82.56 ms, so virtual dispatch was about 9% slower in this test. The difference is small compared with the total work.
- The `std::variant` version was the fastest here (78.27 ms for 10,000,000 bills). A likely reason is that its objects are stored contiguously and are smaller, so less memory is read.
- For 1,000 bills all times are only microseconds, so the choice of method has no practical effect at that size.

### B3.3 – Cost of Exceptions

The validation performance was tested with one million bills using three invalid-data percentages. An invalid bill has a negative amount. The return-code version returns a status; the exception version throws `invalid_argument` and the caller catches it.

| Invalid percentage | Number of valid bills | Number of invalid bills |
|---|---|---|
| 0% | 1,000,000 | 0 |
| 1% | 990,000 | 10,000 |
| 50% | 500,000 | 500,000 |

## Program: Task06/B3_Exception.cpp

```
#include <iostream>
#include <vector>
#include <stdexcept>
#include <chrono>
using namespace std;
using Clock = chrono::steady_clock;

// Return-code version
enum Status { SUCCESS, INVALID_AMOUNT };

__attribute__((noinline))
Status validateRC(double amount, double& out) {
    if (amount < 0) return INVALID_AMOUNT;
    out = amount;
    return SUCCESS;
}

// Exception version
__attribute__((noinline))
double validateEx(double amount) {
    if (amount < 0) throw invalid_argument("negative");
    return amount;
}

int main() {
    const size_t N = 1000000;
    const int percents[] = {0, 1, 50};

    cout << "Validation of " << N << " bills" << endl;
    for (int pct : percents) {
        // pct % of the bills are invalid (negative amount)
        vector<double> amounts(N);
        size_t invalid = 0;
        for (size_t i = 0; i < N; i++) {
            bool bad = false;
            if (pct == 1)  bad = (i % 100 == 0);
            if (pct == 50) bad = (i % 2 == 0);
            amounts[i] = bad ? -100.0 : 1000.0 + (i % 500);
            if (bad) invalid++;
        }

        // return codes
        auto t0 = Clock::now();
        double sumRC = 0; size_t badRC = 0;
        for (double a : amounts) {
            double v;
            if (validateRC(a, v) == SUCCESS) sumRC += v; else badRC++;
        }
```

```
        auto t1 = Clock::now();

        // exceptions
        auto t2 = Clock::now();
        double sumEx = 0; size_t badEx = 0;
        for (double a : amounts) {
            try { sumEx += validateEx(a); }
            catch (const invalid_argument&) { badEx++; }
        }
        auto t3 = Clock::now();

        double msRC = chrono::duration<double, milli>(t1 - t0).count();
        double msEx = chrono::duration<double, milli>(t3 - t2).count();
        cout << pct << "% invalid (" << invalid << " bad): return-code = " << msRC
             << " ms, exception = " << msEx << " ms"
             << "  [check: " << badRC << "/" << badEx << " rejected, sums equal = "
             << (sumRC == sumEx ? "yes" : "NO") << "]" << endl;
    }
    return 0;
}
```

```
G++ - B3.3 return codes vs exceptions (-O2)

$ g++ -std=c++17 -O2 -Wall -Wextra B3_Exception.cpp -o B3_Exception && ./B3_Exception
Validation of 1000000 bills
0% invalid (0 bad): return-code = 1.03965 ms, exception = 2.5895 ms
      [check: 0/0 rejected, sums equal = yes]
1% invalid (10000 bad): return-code = 1.23687 ms, exception = 13.3641 ms
      [check: 10000/10000 rejected, sums equal = yes]
50% invalid (500000 bad): return-code = 0.997518 ms, exception = 607.036 ms
      [check: 500000/500000 rejected, sums equal = yes]
```

*Output Screenshot: B3_Exception.cpp (g++ -O2)*

**Experimental Results Table (time in milliseconds):**

| Invalid percentage | Return-code time (ms) | Exception time (ms) |
|---|---|---|
| 0% | 1.04 | 2.59 (2.5× slower) |
| 1% | 1.24 | 13.36 (10.8× slower) |
| 50% | 0.9975 | 607.0 (608.5× slower) |

**Observations:**

- With no invalid data both methods finish in a few milliseconds for a million bills.

- At 1% invalid data (10,000 throws) the exception version needed 13.36 ms against 1.24 ms for return codes.
- At 50% invalid data (500,000 throws) the exception version needed 607.0 ms, about 608.5 times the return-code time. Each throw costs roughly 1.2 microseconds.
- Throwing is expensive only when it actually happens, so exceptions suit rare errors and return codes suit frequent, expected invalid input.

### B3.4 – Engineering Trade-Off

- Virtual functions provide a flexible and extensible design, at a small measured speed cost.
- Switch statements are simple for a small and fixed number of bill types.
- Exceptions separate error handling from normal program logic.
- Return codes are useful when invalid data is frequent.
- Performance depends on compiler optimization, hardware, memory layout and test data.

### *Alternative Concept – std::variant*

`std::variant` can store one of several bill types in a single container. The `std::visit` function processes the active bill type. It is implemented and timed in `B3_Variant.cpp` above. A small example:

```
#include <iostream>
#include <variant>
using namespace std;

struct OutPatientBill {
    double consultation, tests;
    double total() const { return consultation + tests; }
};

struct InPatientBill {
    int days;
    double rate, surgery;
    double total() const { return days * rate + surgery; }
};
```

```
int main() {
    variant<OutPatientBill, InPatientBill> bills[] = {
        OutPatientBill{500, 1200},
        InPatientBill{3, 2500, 40000}
    };


    double sum = 0;
    for (const auto& b : bills) {
        sum += visit([](const auto& x) { return x.total(); }, b);
    }


    cout << "Total: " << sum << endl;
    return 0;
}
```

**Output:** `Total: 49200`

| Method | Time complexity | Main advantage |
|---|---|---|
| Virtual functions | O(n) | Easy extension |
| Switch statement | O(n) | Simple implementation |
| std::variant | O(n) | Type-safe alternative without base-class pointers |

The space required by `std::variant` is based on the largest alternative type plus a discriminator, with implementation-dependent alignment and padding.

**Conclusion:** Virtual functions are suitable for extensible object-oriented systems. Switch statements are suitable for fixed bill types, while `std::variant` is useful when the set of types is known at compile time.

## TASK B4 – FINAL ENGINEERING DECISION

### B4.1 – Implement a Practical Extension

A new bill type called `DentalBill` and a new payer called `CorporatePayer` are added to the complete hospital billing system. The new classes extend the existing design without changing the previously implemented bill and payer classes. The

complete program below contains all five bill types, all four payers, the exception classes and the tests.

**Program: Task07/B4_Extension.cpp**

```
#include <iostream>
#include <iomanip>
#include <memory>
#include <string>
#include <vector>
#include <algorithm>
#include <stdexcept>
using namespace std;

// ---------------- Exceptions ----------------
class BillingError : public runtime_error {
public:
    using runtime_error::runtime_error;
};
class ClaimError : public BillingError {
public:
    using BillingError::BillingError;
};
class PackageError : public BillingError {
public:
    using BillingError::BillingError;
};

// ---------------- Bill hierarchy ----------------
class Bill {
protected:
    string patient;
public:
    explicit Bill(const string& p) : patient(p) {}
    virtual ~Bill() {}
    virtual double total() const = 0;
    virtual string type() const = 0;
    void print() const { cout << left << setw(10) << type() << setw(8) << patient
                              << right << setw(10) << total() << endl; }
protected:
    static void check(double amount) {
        if (amount < 0) throw invalid_argument("negative charge");
    }
};

class OutPatientBill : public Bill {
    double consultation, tests;
public:
    OutPatientBill(const string& p, double c, double t)
        : Bill(p), consultation(c), tests(t) {
        check(c); check(t);
    }
    double total() const override { return consultation + tests; }
    string type() const override { return "OP"; }
};

class InPatientBill : public Bill {
    int days; double rate, procedures, medicines;
public:
    InPatientBill(const string& p, int d, double r, double pr, double m)
        : Bill(p), days(d), rate(r), procedures(pr), medicines(m) {
        check(r); check(pr); check(m);
```

```
    }
    double total() const override { return days * rate + procedures + medicines; }
    string type() const override { return "IP"; }
};

class DayCareBill : public Bill {
    double package;
public:
    DayCareBill(const string& p, double pk) : Bill(p), package(pk) { check(pk); }
    double total() const override { return package; }
    string type() const override { return "DayCare"; }
};

class EmergencyBill : public Bill {
    double base, hours, rate;
public:
    EmergencyBill(const string& p, double b, double h, double r)
        : Bill(p), base(b), hours(h), rate(r) {
        check(b); check(h); check(r);
    }
    double total() const override { return base + hours * rate; }
    string type() const override { return "Emergency"; }
};

// ---------------- NEW bill type (extension) ----------------
class DentalBill : public Bill {
    double consultation, treatment;
public:
    DentalBill(const string& p, double c, double t)
        : Bill(p), consultation(c), treatment(t) {
        check(c); check(t);
    }
    double total() const override { return consultation + treatment; }
    string type() const override { return "Dental"; }
};

// ---------------- Payer hierarchy ----------------
class Payer {
public:
    virtual double amountCovered(const Bill& b) const = 0;
    virtual string name() const = 0;
    virtual ~Payer() {}
};

class SelfPay : public Payer {
public:
    double amountCovered(const Bill& b) const override { return b.total(); }
    string name() const override { return "SelfPay"; }
};

class Insurance : public Payer {
    double limit, copay;
public:
    Insurance(double l, double c) : limit(l), copay(c) {}
    double amountCovered(const Bill& b) const override {
        return min(b.total(), limit) * (1.0 - copay);
    }
    string name() const override { return "Insurance"; }
};

class GovernmentScheme : public Payer {
    static double rate(const string& t) {
        if (t == "OP") return 1500;
```

```
        if (t == "IP") return 50000;
        if (t == "DayCare") return 15000;
        if (t == "Emergency") return 20000;
        throw PackageError("no government package for bill type " + t);
    }
public:
    double amountCovered(const Bill& b) const override {
        return min(b.total(), rate(b.type()));
    }
    string name() const override { return "GovtScheme"; }
};

// ---------------- NEW payer (extension) ----------------
class CorporatePayer : public Payer {
public:
    double amountCovered(const Bill& b) const override { return b.total() * 0.80; }
    string name() const override { return "Corporate"; }
};

// ---------------- Claim approval ----------------
double approveClaim(const Bill& b, const Payer& p, double cover) {
    double claim = p.amountCovered(b);
    if (claim > cover)
        throw ClaimError("claim " + to_string((long)claim) +
                         " exceeds cover " + to_string((long)cover));
    return claim;
}

int main() {
    cout << fixed << setprecision(0);

    // All objects are owned by smart pointers (RAII)
    vector<unique_ptr<Bill>> bills;
    bills.push_back(make_unique<OutPatientBill>("Ravi", 500, 1200));
    bills.push_back(make_unique<InPatientBill>("Meena", 3, 2500, 40000, 0));
    bills.push_back(make_unique<DayCareBill>("Kiran", 18000));
    bills.push_back(make_unique<EmergencyBill>("Anita", 3000, 4, 1500));
    bills.push_back(make_unique<DentalBill>("Suresh", 800, 6200));

    vector<unique_ptr<Payer>> payers;
    payers.push_back(make_unique<SelfPay>());
    payers.push_back(make_unique<Insurance>(50000, 0.10));
    payers.push_back(make_unique<GovernmentScheme>());
    payers.push_back(make_unique<CorporatePayer>());

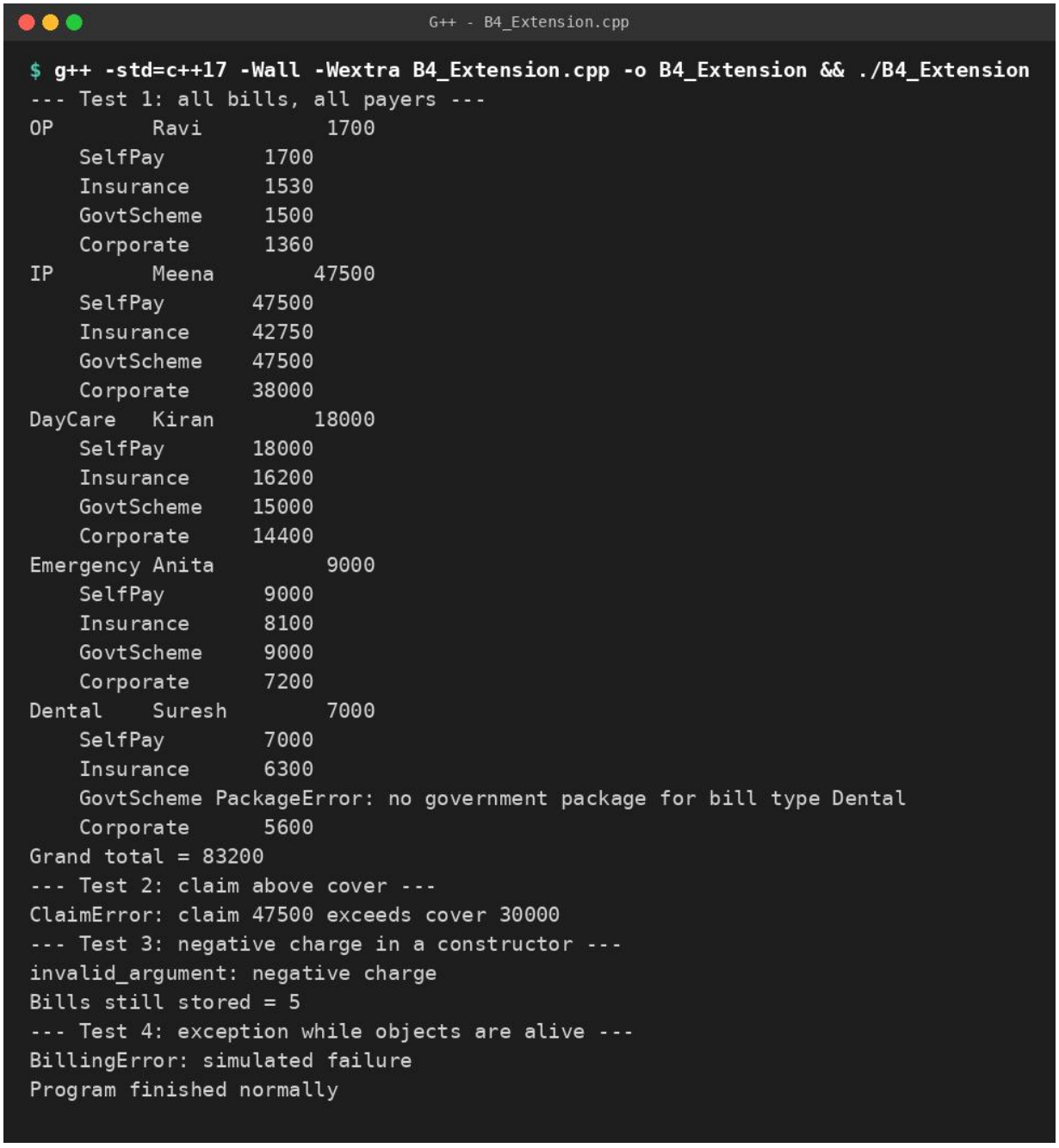
    cout << "--- Test 1: all bills, all payers ---" << endl;
    double sum = 0;
    for (const auto& b : bills) {
        b->print();
        sum += b->total();
        for (const auto& p : payers) {
            try {
                double covered = p->amountCovered(*b);   // may throw here
                cout << "    " << left << setw(11) << p->name()
                     << right << setw(8) << covered << endl;
            }
            catch (const PackageError& e) {
                cout << "    " << left << setw(11) << p->name()
                     << "PackageError: " << e.what() << endl;
            }
        }
    }
    cout << "Grand total = " << sum << endl;
```

```
    cout << "--- Test 2: claim above cover ---" << endl;
    try { approveClaim(*bills[1], *payers[0], 30000); }
    catch (const ClaimError& e) { cout << "ClaimError: " << e.what() << endl; }

    cout << "--- Test 3: negative charge in a constructor ---" << endl;
    try {
        bills.push_back(make_unique<DentalBill>("Bad", -100, 500));
    }
    catch (const invalid_argument& e) {
        cout << "invalid_argument: " << e.what() << endl;
    }
    cout << "Bills still stored = " << bills.size() << endl;

    cout << "--- Test 4: exception while objects are alive ---" << endl;
    try {
        auto temp = make_unique<DentalBill>("Temp", 100, 200);
        throw BillingError("simulated failure");
    }
    catch (const BillingError& e) { cout << "BillingError: " << e.what() << endl; }

    cout << "Program finished normally" << endl;
    return 0;
}
```




*Output Screenshot: B4_Extension.cpp*

**Explanation of the tests:**

- Test 1 processes every bill with every payer through the base-class interfaces. The new `DentalBill` and `CorporatePayer` work without any change to existing classes. The government scheme has no package for dental bills, so it throws `PackageError`, which is caught and reported.
- Test 2 shows `ClaimError` when a claim (47500) exceeds the cover (30000).

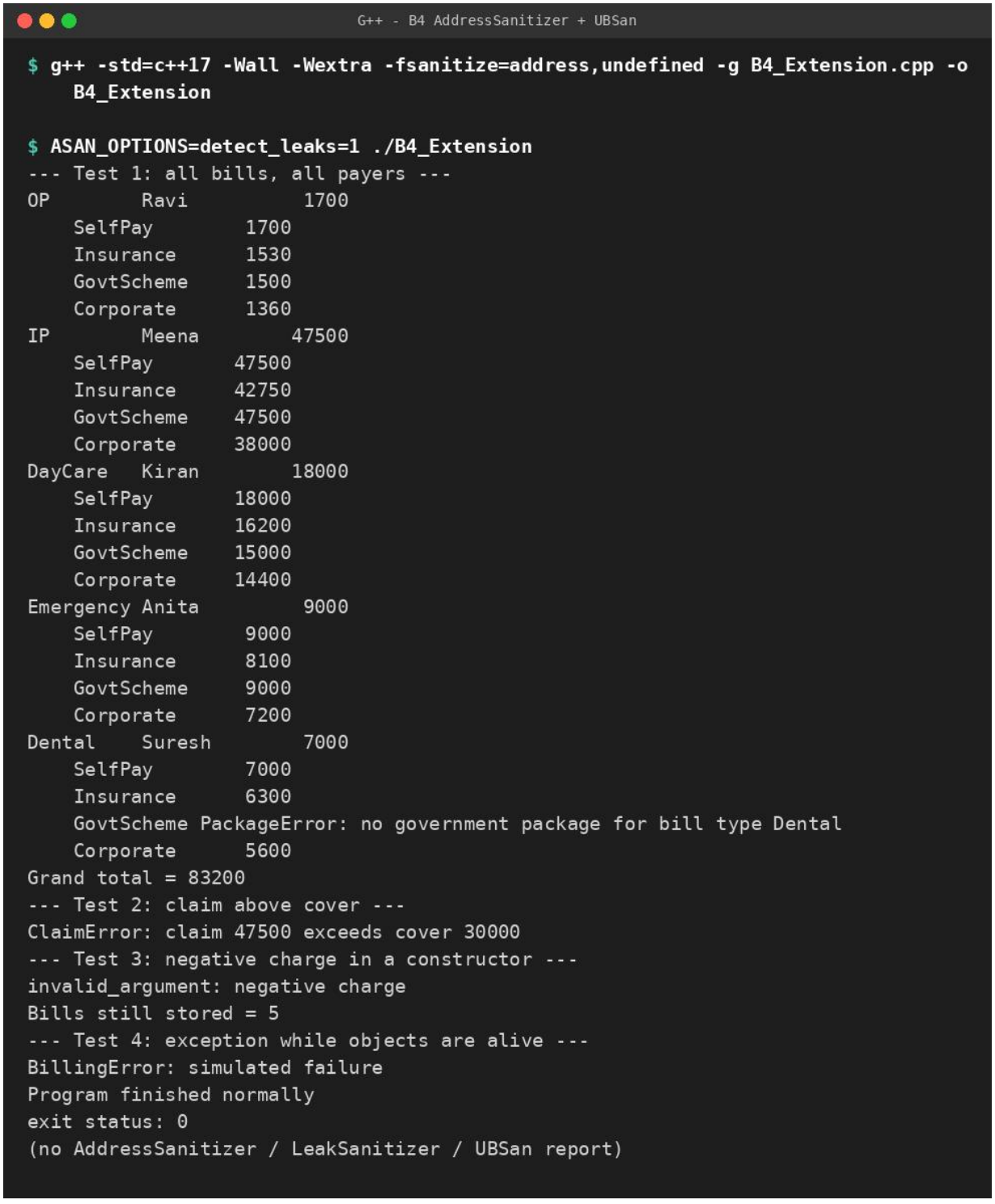
- Test 3 shows that a negative charge is rejected by the constructor with `invalid_argument`; the partly built object is destroyed and the five stored bills are unaffected.
- Test 4 throws an exception while a `unique_ptr`-owned object is alive; the object is released automatically.

The grand total of the five bills is 1700 + 47500 + 18000 + 9000 + 7000 = 83200, matching the program output.

### B4.2 – Check Resource Safety

AddressSanitizer is used to identify memory errors such as memory leaks, invalid memory access and double deletion. The final program, including its exception-handling tests, was compiled with AddressSanitizer and UndefinedBehaviorSanitizer and run with leak detection enabled.

```
g++ -std=c++17 -Wall -Wextra -fsanitize=address,undefined -g B4_Extension.cpp -o
B4_Extension
ASAN_OPTIONS=detect_leaks=1 ./B4_Extension
```




*Output Screenshot: B4_Extension.cpp under AddressSanitizer + UBSan*

**Result:** the program finished with exit status 0 and no AddressSanitizer, LeakSanitizer or UBSan report. (The leak detector was also checked on a deliberately leaking test program, where it reported the leak, so a clean result is meaningful.)

### B4.3 – Production Decision

For a hospital group operating 15 hospitals and processing 200,000 bills per day, the polymorphic design with exception handling is suitable.

**Reasons:**

1. Runtime polymorphism supports different types of bills through a common interface.
2. New bill types (such as `DentalBill`) can be added using derived classes.
3. New payer types (such as `CorporatePayer`) can be added without modifying existing payer classes.
4. Exception handling reports invalid claims and billing errors clearly.
5. Virtual destructors and RAII help prevent resource-management problems, confirmed by a clean sanitizer run.
6. The design is maintainable and reusable across multiple hospitals.

**Performance check using the measurements:** processing one million bills with virtual functions took 9.17 ms (about 9 nanoseconds per bill), so 200,000 bills need only about 1.8 ms. Even with 50% invalid claims, the exception-based validation of 200,000 bills would take roughly 121 ms. The speed difference between virtual functions, switch and variant is therefore not a deciding factor at this scale; database access and network delays will dominate. These figures cover the calculation only.

**Final Decision:** I recommend the polymorphic billing design with exception handling because it provides extensibility, maintainability and safe resource management, and the measured cost is negligible for 200,000 bills per day.

#### *Alternative Concept – Plug-in Configuration Files*

Instead of creating new C++ classes for every bill type and payer, the hospital can store rates and payment rules in configuration files.

| Feature | C++ class extension | Plug-in configuration |
|---|---|---|
| Extension | Add derived classes | Add configuration rules |
| Processing | Virtual function call | Rule interpretation |
| Time | O(1) per bill calculation, excluding calculation work | Depends on rule lookup and interpretation |
| Space | Object data and polymorphic overhead | Configuration rules and runtime storage |
| Flexibility | Requires code changes and compilation | Rules can be updated through configuration |
| Risk | Programming and integration errors | Invalid configuration and rule errors |

**Conclusion:** C++ class-based extensions are suitable for complex behaviour. Configuration files are useful when billing rates and rules change frequently. A combination of both approaches can provide flexibility and maintainability.